

WORLD OF WARCRAFT CLASSIC

# Gear Build Calculator

---

*Product and Engineering Roadmap for Claude Code*

## **Product mission**

Build a clean, trustworthy workspace that lets a player choose a Classic ruleset, class, role, and build profile; enter an item manually or find it in an item database; and receive an explainable comparison showing which item better supports that build and why.

## **Core promise**

Do the game math once, make the assumptions visible, and never hide a recommendation behind a mystery score.

Planned as a local-first Python application with a polished web UI and provider-independent item data layer.

# 1. Executive Summary

The calculator should behave like a lightweight theorycrafting workbench rather than a static best-in-slot list. The user supplies context first - ruleset, level, class, role, specialization/build profile, content type, and optional current stats - because the value of an item changes with those assumptions.

## Critical design decision

Classic WoW is not one immutable ruleset. Era, seasonal realms, progression versions, patch/phase, PvE vs PvP, faction/race, and level can change item availability or stat value. Never hard-code one global scoring table. Treat rulesets and build profiles as versioned configuration.

## The end-to-end user flow

```
graph TD
    A[Select ruleset / phase / level] --> B[Select class -> role -> build profile]
    B --> C[Enter current stats (optional but recommended)]
    C --> D[Search item OR manually enter item stats]
    D --> E[Normalize item and apply eligibility rules]
    E --> F[Evaluate caps, thresholds, derived stats, procs, and set context]
    F --> G[Return score + deltas + explanation + uncertainty]
    G --> H[Compare / save / build a gear set]
```

## What Version 1 must accomplish

- Manual item entry with validation.
- Item search/lookup through a provider adapter, with a local cache and bundled fallback dataset.
- Class, role, and build-profile selection.
- Role-aware scoring with configurable stat weights, caps, and thresholds.
- Side-by-side item comparison with explainable math.
- A clean, fast UI that works well before advanced simulation exists.
- Automated tests for scoring, validation, data normalization, and the most important UI flows.

## Roadmap at a Glance

| Version | Primary goal            | User question                             |
|---------|-------------------------|-------------------------------------------|
| 0.1     | Foundation and rulesets | What assumptions does the calculator use? |

| Version | Primary goal                    | User question                                                    |
|---------|---------------------------------|------------------------------------------------------------------|
| 1.0     | Item calculator MVP             | Which of these items is better for my build?                     |
| 1.5     | Profiles and gear sets          | How does this item affect my current setup?                      |
| 2.0     | Advanced Classic math           | How do caps, weapon rules, procs, and set bonuses change value?  |
| 3.0     | Gear optimizer                  | What combination of items best satisfies my build constraints?   |
| 4.0     | Encounter-aware recommendations | Does the recommendation change for this fight or content type?   |
| 5.0     | Validation and simulation       | Do the scoring assumptions hold up against simulations and logs? |

## 2. Product Scope and User Personas

### Primary user stories

- As a Warrior tank, I want to compare two shields and understand the tradeoff between mitigation, threat, and survivability.
- As a Holy Paladin, I want to compare healing gear while respecting mana sustainability and spell-healing priorities.
- As a Rogue DPS player, I want hit, crit, attack power, weapon speed, and weapon-skill context to affect item value instead of using generic stat weights.
- As a leveling player, I want the same calculator to work at my current level without assuming raid-level caps.
- As a theorycrafter, I want to inspect and override stat weights rather than being forced to trust hidden defaults.
- As a casual user, I want a simple answer first, with advanced assumptions available when I choose to inspect them.

### Initial supported class/role matrix

| Class   | Initial roles / profiles            |
|---------|-------------------------------------|
| Warrior | Tank; Arms DPS; Fury DPS            |
| Paladin | Healer; Tank; Melee DPS             |
| Druid   | Tank; Healer; Melee DPS; Caster DPS |
| Priest  | Healer; Shadow DPS                  |

| Class   | Initial roles / profiles               |
|---------|----------------------------------------|
| Shaman  | Healer; Enhancement DPS; Elemental DPS |
| Rogue   | Melee DPS                              |
| Hunter  | Ranged physical DPS                    |
| Mage    | Caster DPS                             |
| Warlock | Caster DPS                             |

### Build profile, not just class + role

Two characters can share the same class and role while valuing stats differently. The UI should therefore select a named build profile after class and role. Examples: Warrior "Deep Prot" vs "Fury/Prot" tanking; Paladin "Raid Holy" vs a leveling healer profile. Profiles are data/configuration, not branches of UI code.

### Ruleset dimensions that must be explicit

- Game family / ruleset: Classic Era first; later add other Classic variants through adapters and config.
- Patch or phase: controls item availability, set bonuses, and balance assumptions.
- Character level: affects rating/cap logic and item eligibility.
- Faction and race: needed when racial abilities or weapon skill affect comparisons.
- Content mode: raid PvE, dungeon PvE, leveling, solo, or PvP.
- Encounter context: optional later; e.g., physical-heavy tank fight, resistance requirement, or short burn phase.

## 3. Proposed System Architecture

External item providers / bundled data / manual entry



```
presentation models
|
Streamlit UI
```

## Architectural rules

- Data-source adapters fetch; they do not score items.
- Normalization converts provider-specific fields into one canonical Item model.
- Rulesets define game/version assumptions; build profiles define role priorities.
- The calculation engine owns all formulas. The UI never performs theorycraft math.
- Presentation code formats results; it does not change them.
- A recommendation must expose its assumptions, caps, penalties, and uncertainty.
- Manual items and looked-up items must pass through the exact same scoring pipeline.
- Every score must be reproducible from stored inputs and a versioned scoring profile.

## Recommended technology stack

| Layer              | Recommendation      | Reason                                                                        |
|--------------------|---------------------|-------------------------------------------------------------------------------|
| Language           | Python 3.12+        | Excellent for rules engines, data validation, testing, and rapid development. |
| Models             | Pydantic            | Strong validation and explicit schemas for items, profiles, and results.      |
| Storage            | SQLite + SQLAlchemy | Simple local persistence with a clean upgrade path.                           |
| UI                 | Streamlit           | Fastest route to a polished Python-first research/calculator workspace.       |
| Interactive charts | Plotly              | Useful later for stat tradeoffs, sensitivity, and gear-set comparisons.       |
| HTTP               | httpx               | Clean async-capable provider clients.                                         |
| Testing            | pytest              | Unit, integration, regression, and calculation fixture testing.               |

## 4. Repository Structure

```
wow-classic-gear-calculator/
├─ README.md
├─ CLAUDE.md
├─ pyproject.toml
├─ requirements.txt
├─ .env.example
├─ configs/
```

```

├── app.yaml
├── providers.yaml
├── rulesets/
│   └── classic_era.yaml
├── profiles/
│   ├── warrior/
│   ├── paladin/
│   ├── druid/
│   └── ...
├── data/
│   ├── raw/
│   ├── processed/
│   ├── cache/
│   └── fixtures/
├── src/wow_gear/
│   ├── core/
│   ├── models/
│   ├── data_sources/
│   ├── processing/
│   ├── repositories/
│   ├── rulesets/
│   ├── profiles/
│   ├── calculations/
│   ├── scoring/
│   ├── comparison/
│   ├── optimizer/
│   ├── services/
│   └── reporting/
├── apps/
│   ├── streamlit_app/
│   │   ├── app.py
│   │   ├── pages/
│   │   ├── components/
│   │   └── state/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── regression/
│   └── ui/
├── docs/
└── scripts/

```

## Folder responsibilities

| Package      | Owns                                                                                        |
|--------------|---------------------------------------------------------------------------------------------|
| models       | Canonical Item, CharacterContext, BuildProfile, ScoreResult, GearSet, and provider schemas. |
| data_sources | External API/database clients and bundled data readers.                                     |

| Package      | Owns                                                                                             |
|--------------|--------------------------------------------------------------------------------------------------|
| processing   | Normalization, deduplication, text/stat parsing, and item validation.                            |
| rulesets     | Version-specific game mechanics, caps, allowed stats, item availability, and formula parameters. |
| profiles     | Class/role/build priorities and default weights.                                                 |
| calculations | Derived stats, caps, weapon math, mitigation, mana, threat, and proc math.                       |
| scoring      | Turns normalized contribution into comparable build-specific values.                             |
| comparison   | Item A vs Item B and replacement-delta logic.                                                    |
| optimizer    | Whole-gear-set search and constraints in later versions.                                         |
| services     | Use cases consumed by UI, CLI, or future API.                                                    |

## 5. Core Data Model

### Canonical Item

```

Item
- id / external_ids
- name
- ruleset
- phase
- required_level
- slot
- armor_type / weapon_type
- armor
- strength / agility / stamina / intellect / spirit
- attack_power / ranged_attack_power
- spell_power / healing_power
- hit / spell_hit / crit / spell_crit
- defense / block / dodge / parry
- resistances
- weapon_min_damage / weapon_max_damage / speed / dps
- equip_effects[]
- on_use_effects[]
- set_id / set_name
- source / provider / source_url
- data_version / fetched_at

```

## CharacterContext

```
CharacterContext
- ruleset / patch / phase
- level
- class_name
- role
- build_profile_id
- faction / race
- content_mode
- current_stats (optional)
- current_gear (optional)
- buffs / consumables assumptions (optional)
- encounter_profile (optional)
```

## BuildProfile

```
BuildProfile
- profile_id
- class_name / role / specialization_label
- scoring_model_version
- stat_weights
- hard_caps
- soft_caps
- threshold_rules
- weapon_preferences
- derived_stat_rules
- proc_assumptions
- set_bonus_rules
- default_content_mode
- notes / citations / validation_status
```

### Do not store one universal "item score" on the item

An item has raw properties. A score is the result of evaluating those properties against a specific CharacterContext and BuildProfile. The same item can legitimately receive very different scores for a Warrior tank and a Holy Paladin.

## 6. Scoring and Calculation Engine

Version 1 should use an explainable weighted model, but the engine must be structured so later formulas can replace simple weights where Classic mechanics require them.

### Baseline score

```
raw_score = sum(normalized_stat_value(stat) * profile_weight(stat))

final_score = raw_score
+ threshold_adjustments
```

- + weapon\_adjustments
- + proc\_value
- + set\_bonus\_delta
- + context\_adjustments

## The calculator must model diminishing or zero value past caps

| Rule type          | Example behavior                                                                   |
|--------------------|------------------------------------------------------------------------------------|
| Hard cap           | Value falls to zero after the relevant cap for the selected ruleset/build.         |
| Soft cap           | Value decreases after a threshold rather than disappearing.                        |
| Breakpoint         | Crossing a threshold produces a discrete change in value.                          |
| Conditional stat   | Value depends on weapon type, current gear, encounter, or active buff assumptions. |
| Mutually exclusive | Two effects cannot both be counted at full value under the same assumption.        |

## Role-specific calculation families

| Role         | Examples of important derived outputs                                                                                  |
|--------------|------------------------------------------------------------------------------------------------------------------------|
| Physical DPS | Attack power contribution, hit/crit context, weapon DPS/speed, weapon skill, expected damage contribution.             |
| Caster DPS   | Spell power, hit/crit context, school-specific assumptions, mana sustainability, cast pattern.                         |
| Tank         | Effective health, mitigation, avoidance, block, defense thresholds, threat contribution, resistance context.           |
| Healer       | Healing throughput, mana pool, regeneration/sustainability, crit, spell/healing power, encounter duration assumptions. |
| PvP          | Stamina/survivability, burst, control/resource assumptions, resistances; separate profile from PvE.                    |

### Explainability requirement

Every comparison should show the score delta by component. Example: +12.4 from healing power, +5.1 from intellect, -3.8 from lost regeneration, and 0 from excess hit beyond the selected cap. Users should be able to see why the recommendation changed.

## 7. Item Entry and Lookup

### Path A - Manual item entry

- User selects item slot and item type first so the form only displays relevant fields.
- Stat fields accept numeric values; optional effects can be entered as structured effect rows.
- Validation warns about impossible combinations but still supports a custom/theorycrafted item mode.
- The normalized item is previewed before scoring.

### Path B - Item search

- Search by item name with typeahead suggestions.
- Allow exact item ID lookup for advanced users.
- Filter results by ruleset, phase, slot, armor/weapon type, and required level.
- Cache looked-up items locally so repeat searches remain fast and provider outages do not break basic use.

### Provider strategy

| Priority  | Source strategy                        | Use                                                                                                            |
|-----------|----------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 1         | Bundled / curated Classic item dataset | Stable baseline and offline fallback.                                                                          |
| 2         | Warcraft Logs API adapter              | Useful for Classic game data and item identifiers; cache static game data aggressively.                        |
| 3         | Blizzard developer API adapter         | Official-source option where Classic coverage fits the required item fields.                                   |
| 4         | Manual import / CSV / JSON             | Lets users add missing or seasonal items without waiting for provider support.                                 |
| Reference | Wowhead Classic database               | Excellent human reference; do not make scraping it a core dependency without explicit permission/terms review. |

#### Provider independence

The UI should call ItemSearchService, never Warcraft Logs, Blizzard, or another site directly. A provider can fail, change fields, or disappear without forcing a rewrite of the calculation engine or UI.

### Canonical normalization pipeline

Provider response / manual form

-> provider parser

-> canonical Item model

- > validation
- > item repository/cache
- > scoring service

## 8. Clean UI / UX Specification

The UI should feel like a polished calculator first and a theorycrafting lab second. Advanced controls should be available without overwhelming a first-time user.

### Primary navigation

| Page                   | Purpose                                                                   |
|------------------------|---------------------------------------------------------------------------|
| Calculator             | Choose build context, add/search items, and get a recommendation.         |
| Item Compare           | Compare two or more items with detailed score deltas.                     |
| Build Profiles         | Inspect or customize stat weights, caps, assumptions, and notes.          |
| Gear Set               | Assemble current gear and see aggregate stats / weak slots.               |
| Optimizer              | Later: search combinations under constraints.                             |
| Item Database          | Browse cached items and provider/source metadata.                         |
| Settings / Data Health | Ruleset selection, provider status, refresh, cache version, and warnings. |

### Calculator page layout

|                                                        |                 |       |  |
|--------------------------------------------------------|-----------------|-------|--|
| +-----+                                                |                 |       |  |
| CLASS / ROLE / BUILD                                   | RULESET / PHASE | LEVEL |  |
| +-----+                                                |                 |       |  |
| Current context: Warrior > Tank > Fury/Prot > Raid PvE |                 |       |  |
| +-----+                                                |                 |       |  |
| ITEM INPUT                                             | RECOMMENDATION  |       |  |
| [ Search items... ]                                    | Item A          | 84.2  |  |
| or [ Enter stats manually ]                            | Item B          | 77.6  |  |
|                                                        | Delta           | +6.6  |  |
| Item A                                                 |                 |       |  |
| Item B                                                 | Why?            |       |  |
|                                                        | + mitigation    |       |  |
|                                                        | + stamina       |       |  |
|                                                        | - threat        |       |  |
| +-----+                                                |                 |       |  |
| Advanced assumptions   caps   current stats   raw math |                 |       |  |
| +-----+                                                |                 |       |  |

## Visual design system

- Light neutral background with restrained navy/charcoal surfaces and warm gold accents inspired by Classic WoW without imitating the game UI.
- Green only for beneficial deltas; red only for harmful deltas; gold for selected/recommended state; never rely on color alone.
- Large primary result, short explanation, then expandable detailed math.
- Compact stat chips for item summaries; consistent alignment for numbers.
- Use item-quality colors only as secondary metadata, not as the main score signal.
- Responsive layout with a single-column fallback for narrow screens.
- Loading, empty, stale-data, provider-error, and invalid-item states designed explicitly.

## 9. Version Roadmap

### V0.1 - Foundation

- Repository, environment, configuration, logging, and testing setup.
- Canonical models for Item, CharacterContext, BuildProfile, and ScoreResult.
- Ruleset loader and one Classic Era ruleset configuration.
- At least four representative build profiles: physical DPS, caster DPS, healer, tank.
- Fixture items with hand-calculated expected results.

### V1.0 - Gear Calculator MVP

- Manual item-entry flow.
- Search/lookup flow with one provider plus local cache.
- Class -> role -> build-profile selector.
- Weighted scoring with hard/soft cap hooks.
- Item A vs Item B comparison.
- Transparent score breakdown and assumptions panel.
- Polished Calculator, Compare, Profiles, and Data Health pages.
- Full unit/integration/UI test baseline.

### V1.5 - Character Profiles and Gear Sets

- Save named character/build profiles locally.
- Enter or import a full current gear set.
- Calculate aggregate stats and show how a replacement changes the whole character, not just one slot.
- Identify capped or wasted stats and under-served priorities.
- Export comparison results as JSON/CSV and a shareable report.

### V2.0 - Advanced Classic Theorycraft Math

- Weapon speed/DPS and weapon-skill-aware models where applicable.
- Derived mitigation, avoidance, effective health, healing sustain, and threat calculations.
- Proc/on-use expected-value framework with uptime assumptions.

- Set bonus activation/deactivation deltas.
- Resistance and encounter-specific weighting.
- Sensitivity analysis: show when a recommendation flips as assumptions change.

### **V3.0 - Gear Optimizer**

- Search the local item pool for best combinations by slot.
- Lock/exclude items and enforce phase, class, faction, armor, unique-equip, and set constraints.
- Optimize for a profile objective such as threat, survivability, healing sustain, or balanced score.
- Generate several near-optimal alternatives so the user can account for accessibility and preference.

### **V4.0 - Encounter-aware Recommendations**

- Encounter profiles for short/long fights, tank damage mix, resistance needs, movement, and mana pressure.
- Per-encounter recommendations instead of one global best-in-slot answer.
- Raid-plan mode: compare one gear set across several encounters and optimize for overall utility.

### **V5.0 - Validation, Logs, and Simulation**

- Optional Warcraft Logs integration for empirical validation and gear/performance context.
- Compare predicted stat value with observed or simulated performance.
- Calibrate assumptions per content profile rather than silently changing weights.
- Track confidence/validation status for build profiles and formulas.

## **10. Build Profile Methodology**

The hardest part of this project is not the UI or item lookup. It is defining trustworthy build profiles. Treat each profile as a small, versioned research artifact.

### **Profile creation checklist**

- State the target ruleset, phase, level, content mode, class, role, and build label.
- List every scored stat and its baseline weight.
- List caps, soft caps, breakpoints, and conditions.
- Define current-stat dependencies such as remaining hit needed.
- Document weapon assumptions and eligible weapon types.
- Define proc/on-use uptime assumptions if used.
- Document set-bonus rules and whether partial sets matter.
- Attach sources/notes used to establish the profile.
- Create at least five hand-reviewed item comparisons that the profile should rank correctly.
- Mark the profile as draft, validated, or experimental.

## Recommended score result

```
ScoreResult
- overall_score
- rank / recommendation_label
- component_breakdown[]
- capped_stats[]
- threshold_events[]
- warnings[]
- assumptions[]
- profile_id / profile_version
- ruleset_version
- data_source / item_version
- confidence / validation_status
```

### Never present a score as absolute truth

The UI should say "recommended under this profile and these assumptions". If the user has not supplied current stats, warn that cap-sensitive recommendations may be approximate.

## 11. Testing and Reliability Strategy

### Unit tests

- Every stat conversion and derived formula.
- Hard cap, soft cap, and breakpoint behavior immediately below, at, and above thresholds.
- Manual item normalization and provider item normalization produce equivalent canonical Items.
- Weapon/type eligibility rules.
- Set bonus activation and replacement deltas.
- Profile loader rejects invalid/missing rules.

### Golden calculation fixtures

Create small JSON fixtures containing a context, item(s), and expected component-by-component result. These protect the most important game math from accidental regression.

```
fixture: warrior_tank_shield_comparison_001.json
context: Classic Era / level 60 / tank / profile v1.2
item_a: ...
item_b: ...
expected:
  winner: item_a
  score_delta: ...
  capped_stats: [...]
  warnings: [...]
  component_ranges: {...}
```

## Integration tests

- Provider response -> normalization -> repository -> search result.
- Search result -> score -> comparison output.
- Cache fallback when provider is unavailable.
- Ruleset/profile version changes produce deterministic output.

## UI tests

- User can select class, role, and profile.
- Manual item flow validates and scores.
- Search flow selects an item and scores it.
- Changing current stats can update a cap-sensitive recommendation.
- Provider error produces a usable fallback message instead of a crash.

## Definition of done for every milestone

Implementation complete  
-> targeted tests pass  
-> full suite passes  
-> no provider secrets in repo  
-> manual UI smoke test passes  
-> assumptions documented  
-> git diff reviewed  
-> commit milestone

## 12. Claude Code Build Plan

Do not ask Claude Code to build the entire application in one run. Use bounded milestones with an architecture review and test gate after each milestone.

### Recommended milestone sequence

| Milestone | Deliverable                                                        |
|-----------|--------------------------------------------------------------------|
| 1         | Repository skeleton, CLAUDE.md, configuration, CI/test foundation. |
| 2         | Canonical models + ruleset/profile loaders + fixtures.             |
| 3         | Core scoring engine + caps/threshold hooks + golden tests.         |
| 4         | Manual item-entry service and validation.                          |
| 5         | Item repository + first lookup provider + cache.                   |
| 6         | Item comparison service + explainable breakdown.                   |
| 7         | Streamlit shell + design system + calculator page.                 |
| 8         | Compare, Profiles, Item Database, and Data Health pages.           |

| Milestone | Deliverable                                           |
|-----------|-------------------------------------------------------|
| 9         | Saved character profile + full gear-set context.      |
| 10        | V1 audit: correctness, architecture, UX, tests, docs. |

## Prompt pattern for each milestone

Read CLAUDE.md and inspect the relevant repository files before making changes.

Objective:

[one bounded milestone]

Constraints:

- Preserve the layer boundaries in the architecture.
- Do not duplicate formulas.
- Do not hard-code ruleset values in UI code.
- Do not weaken existing tests to make them pass.
- Keep provider-specific fields out of canonical calculation code.
- Add tests for normal, edge, and failure cases.

Before coding:

1. Explain the implementation plan.
2. Identify files to change.
3. Identify theorycraft assumptions or unknowns.

After coding:

1. Run targeted tests.
2. Run the full suite.
3. Summarize changed files, decisions, assumptions, and follow-up risks.

## 13. Suggested CLAUDE.md Project Constitution

# WoW Classic Gear Calculator Engineering Rules

## Architecture

- Rulesets own version-specific game mechanics.
- Build profiles own class/role priorities and assumptions.
- Calculation modules own formulas.
- Scoring modules combine calculation outputs into comparisons.
- UI code never performs game math.
- Provider adapters never contain scoring logic.

## Data integrity

- Preserve provider IDs and source metadata.
- Normalize all items into the canonical Item model.
- Manual and looked-up items use the same validation/scoring path.
- Ruleset, profile, and item-data versions must be traceable.

### ## Theorycraft correctness

- Never assume one stat weight applies to every class/role.
- Caps and breakpoints are context-dependent.
- Do not silently invent missing proc or set-bonus values.
- Expose assumptions and uncertainty to the user.
- Prefer testable formulas over opaque recommendation logic.

### ## Testing

- Every formula requires deterministic tests.
- Use hand-reviewed golden fixtures for critical comparisons.
- Do not modify expected values merely to make a failing test pass.
- Run the full suite before closing a milestone.

### ## Development

- Inspect existing code before edits.
- Implement the smallest coherent change.
- Avoid unrelated refactors during feature work.
- Update docs when rules, profiles, data sources, or user behavior change.

## 14. Major Risks and How to Design Around Them

| Risk                                         | Mitigation                                                                      |
|----------------------------------------------|---------------------------------------------------------------------------------|
| Classic rules differ by version/phase        | Version every ruleset and profile; require explicit context selection.          |
| Item provider changes or is incomplete       | Provider interface + local cache + bundled dataset + manual entry.              |
| Static weights give bad cap-sensitive advice | Accept current stats and calculate marginal value after caps.                   |
| Procs/set bonuses are hard to value          | Add structured effect model and explicit assumptions; mark unsupported effects. |
| Players treat score as universal truth       | Show component deltas, warnings, validation status, and editable profiles.      |
| Calculator becomes too complicated           | Progressive disclosure: simple default view, expandable advanced math.          |
| UI and formulas drift apart                  | UI consumes service/result models only; calculations are centralized.           |
| Regression after profile changes             | Golden fixtures tied to profile/ruleset versions.                               |

### Out of scope for the first release

- Perfect simulation of every spell rotation or encounter.

- Automatic scraping of websites as a required dependency.
- A universal best-in-slot list for every phase/spec.
- Character armory import as a prerequisite for use.
- AI-generated stat weights without deterministic formulas and validation.

## 15. Version 1 Acceptance Criteria

V1 is complete only when a new user can perform the full core workflow without touching code.

| Area         | Acceptance test                                                                               |
|--------------|-----------------------------------------------------------------------------------------------|
| Context      | User can select Classic Era, phase, level, class, role, and build profile.                    |
| Manual item  | User can enter an item manually and receive validation plus a score.                          |
| Lookup       | User can search/select a known item through the provider/cache layer.                         |
| Comparison   | User can compare at least two items and see a winner, score delta, and component explanation. |
| Caps         | At least one cap-sensitive profile demonstrates reduced/zero marginal value correctly.        |
| Profiles     | User can inspect the profile weights/thresholds used for the result.                          |
| Traceability | Result records ruleset/profile version and item source.                                       |
| Reliability  | Provider failure does not prevent manual comparison or cached-item use.                       |
| UI           | Core pages are visually consistent, responsive, and handle empty/error/loading states.        |
| Tests        | All unit/integration/UI tests pass, including hand-reviewed golden comparisons.               |

### V1 success statement

A player can answer: "For this specific Classic build and my current stat situation, which of these two items is better, by how much, and why?" - using either manual stats or an item lookup - without needing to trust a black-box score.

## 16. Future Gear Optimizer Design

Once single-item scoring is reliable, the optimizer can search whole gear sets. It should optimize under constraints rather than simply pick the highest standalone score in each slot.

### Optimizer inputs

- Ruleset, phase, class, role, build profile, level, faction/race.
- Allowed item sources or content tiers.
- Locked items and excluded items.
- Required set bonuses or minimum set pieces.
- Stat caps/targets and encounter constraints.
- Optimization objective: DPS, threat, mitigation, healing sustain, or balanced composite.
- Accessibility constraints such as "no world bosses" or "dungeon-only".

### Optimizer output

Recommended Set #1  
Overall objective score: 100.0  
Hit target: satisfied  
Set bonuses: 3-piece active  
Warnings: none

Alternative #2  
Objective score: 98.7  
Easier acquisition: yes  
Tradeoff: -1.3 score, +survivability, no rare world-drop dependency

#### Near-optimal alternatives matter

Classic gear acquisition is constrained by raid drops, competition, professions, faction, and time. The best product should show several strong alternatives, not pretend one mathematically highest set is always the practical choice.

## 17. Data Source and Integration Notes

These integrations should be treated as replaceable providers. Verify current terms, authentication requirements, Classic coverage, and field completeness before making any source a production dependency.

| Source               | Roadmap use                                                | Implementation note                                                                                                                             |
|----------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Warcraft Logs API v2 | Classic game data, item identifiers, later log validation. | Uses OAuth 2.0. Classic GraphQL game data exposes static collections including items; cache static game data rather than refetching constantly. |

| Source                  | Roadmap use                                                 | Implementation note                                                                                     |
|-------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Blizzard Developer APIs | Official-source option for supported WoW game/profile data. | Keep behind provider adapter because Classic namespaces/coverage can differ from retail and may evolve. |
| Wowhead Classic DB      | Human research/reference and manual verification.           | Do not make HTML scraping a critical dependency without explicit review of permitted automated access.  |
| Bundled curated data    | Reliable V1 baseline and offline operation.                 | Version the dataset and retain original source provenance.                                              |
| Manual JSON/CSV import  | Escape hatch for missing/seasonal/custom data.              | Validate into the same canonical Item model as every other provider.                                    |

## Reference links for the build team

- Warcraft Logs API documentation: <https://www.warcraftlogs.com/api/docs>
- Warcraft Logs Classic GameData schema: <https://classic.warcraftlogs.com/v2-api-docs/warcraft/gamedata.doc.html>
- Blizzard developer documentation: <https://develop.battle.net/documentation/world-of-warcraft>
- Wowhead Classic database: <https://www.wowhead.com/classic/database>

## 18. First Claude Code Prompt

Use this as the first build instruction after creating the empty repository:

We are starting a new project called WoW Classic Gear Build Calculator.

Read the roadmap and create only Milestone 1: repository foundation.

Goals:

- Establish a src-layout Python project.
- Create the package boundaries described in the roadmap.
- Add CLAUDE.md with the engineering constitution.
- Add configuration directories for rulesets and build profiles.
- Add pytest and a minimal smoke test.
- Add Streamlit as the planned UI dependency, but do not build feature pages yet.
- Add .env.example and a safe .gitignore.
- Create architecture and data-model documentation stubs.

Do not implement scoring formulas, scrape websites, or invent Classic mechanics yet.

Before making changes, show the proposed folder tree and explain any deviation from the roadmap.

After implementation, run the tests and summarize the files created.

## Then build in this order

1. Milestone 2: canonical models + profile/ruleset schemas.

2. Milestone 3: scoring engine with fixture-only data.
3. Milestone 4: manual item entry.
4. Milestone 5: item provider/cache.
5. Milestone 6: comparison/explanation service.
6. Milestone 7: polished UI shell and calculator.
7. Milestone 8: full V1 audit and documentation.

### **Final build principle**

Make the calculator trustworthy before making it powerful. A transparent, tested comparison between two items is a better V1 than an enormous best-in-slot optimizer built on uncertain assumptions.

Roadmap prepared for staged implementation in Claude Code. Keep all theorycraft assumptions versioned, testable, and visible to the user.